

An aerial photograph of a city at sunset. The sky is a mix of blue and orange, with the sun low on the horizon. A rainbow is visible on the left side of the image. In the foreground, there are several modern, multi-story buildings with white facades and many windows. A bridge or walkway connects two of the buildings. In the background, there are more buildings and a tall smokestack emitting a plume of smoke.

Semester Project Information

SWEN3 | BIF5

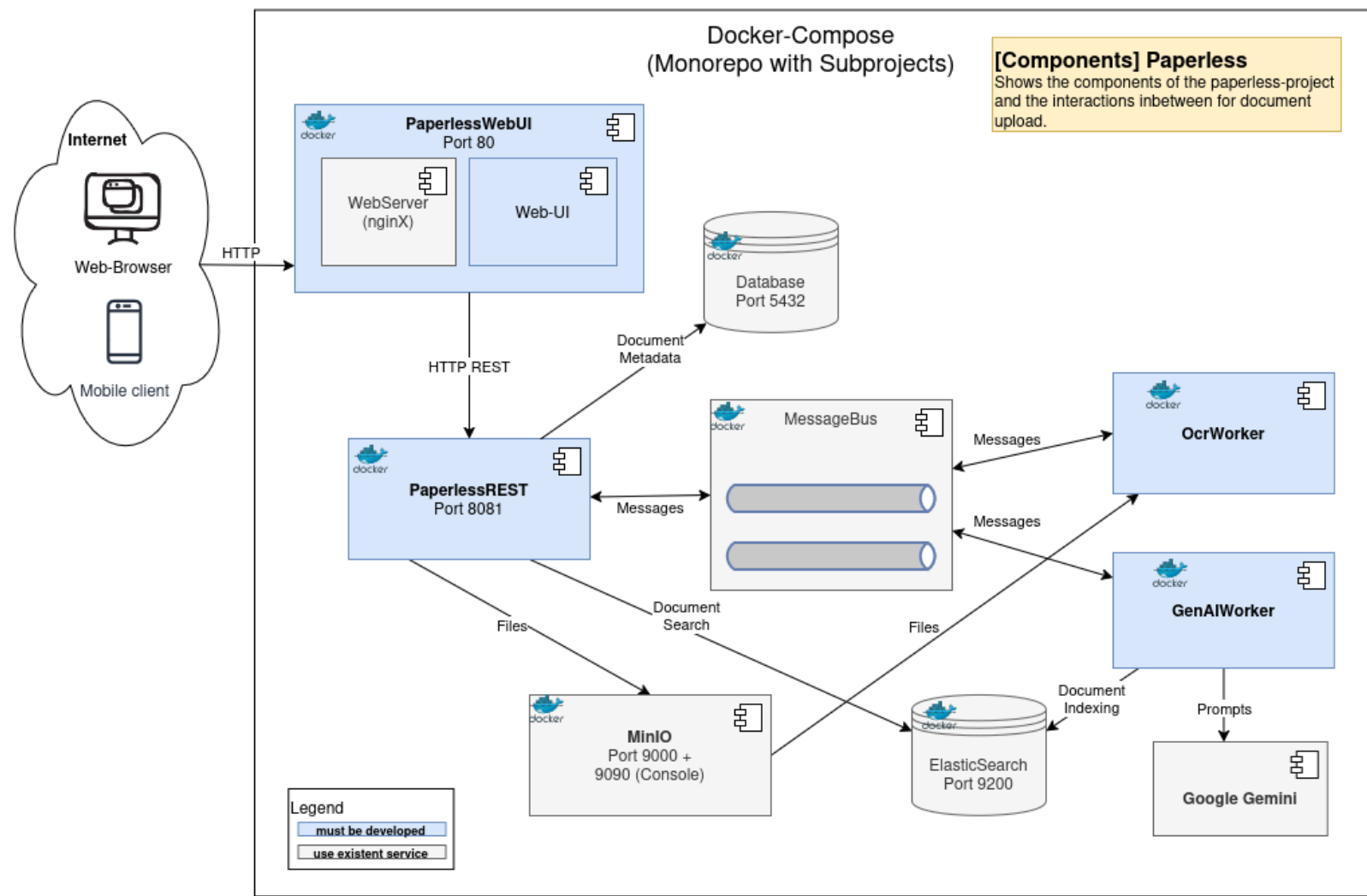
An aerial photograph of a city at sunset. The sun is low on the horizon, casting a warm orange glow over the city. A rainbow is visible in the lower-left corner. The city features a mix of modern and older buildings, with a prominent white building in the foreground. A bridge or walkway connects two parts of the city. In the distance, a tall chimney is visible against the sky.

Semester Project:

Document Management System

*a Document management system for archiving documents in a FileStore,
with automatic OCR (queue for OC-recognition),
automatic summary generation (using Gen-AI),
tagging and full text search (ElasticSearch).*

Semester Project Architecture



Use Cases

1. Upload document

- Automatically performs OCR
- Is indexed for full-text search in Elasticsearch
- a summary is automatically generated

2. Search for a document

- Full-text and fuzzy search in Elasticsearch

3. Manage documents

- Update, delete, metadata

4. Your additional individually defined usecases

- Your own choice
- Must include additional entities!

Technology Stack

Java

- JDK LTS (≥ 25)
- Spring Boot ($\geq 3.5/4$)

C#

- .Net ≥ 10.0
- ASP.Net

Both

- Docker (Desktop)
- RabbitMQ
- PostgreSQL
- ElasticSearch
- GenAI (Google Gemini or similar)

An aerial photograph of a city at sunset. The sky is a mix of blue and orange, with the sun low on the horizon. A rainbow is visible on the left side of the image. In the foreground, there are several modern, multi-story buildings with white facades and many windows. A bridge connects two of these buildings. In the background, a city skyline is visible, including a tall chimney emitting smoke.

Semester Project Sprints

Sprints

Sprint 1	Sprint 2	Sprint 3	Sprint 4	Sprint 5	Sprint 6
Project-Setup	Web-UI	Queuing	Workers	GenAI	Integr.-Test
REST API, DAL			MinIO, OCR, ELK		Batch- Processing
		Mid-Term Code Review			Final Code Review

- **Apply the Software-Development LiveCycle using ALM tools!**
- Use GitHub (Repository) + GitHub Project (Kanban Board)

Sprint 1: Project-Setup, REST API, DAL (with Mapping)

1. Java/C# Project Setup
2. Remote Repository setup,
all team members are able to commit/push
3. REST Server created
Endpoints defined by the team (code-first)
4. ORM is integrated to persist the entities on
the PostgreSQL database, use the repository
pattern
5. Show correct function with unit-tests, mock
out the “production” database
6. Initial docker-compose.yml, used to run the
REST-server & database inside containers
7. Implement your additional use-case in your
project (this must contain additional entities)

Assignment:

- Submit the ZIP-Archive containing the full project **on-time**
(see Moodle Course: „Submission – Sprint 1)
- MUST-HAVE Check Criteria:
 - No build error
 - Servers (REST & PostgreSQL) start successfully
 - REST Endpoints functioning, data is persisted in DB



Sprint 2: (Web-)UI

1. Webserver service (e.g. nginx or else) integrated
2. Implement the UI as WebSite incl. user-input validation
3. Dashboard and detail-pages are served by the webserver
4. The Webpage communication with the REST server
5. Extend docker-compose.yml to run the UI in an additional container

Assignment:

- Submit the ZIP-Archive containing the full project **on-time**
(see Moodle Course: „Submission – Sprint 2)
- MUST-HAVE Check Criteria:
 - No build error (docker compose build)
 - docker compose up successfully starts containers
 - GET <http://localhost/> returns the functioning paperless-frontend



Sprint 3: Queues integration (RabbitMQ)

1. Extend docker-compose.yml to run RabbitMQ in a container
2. Integrate Queues into REST Server
3. on document upload the REST-Server should also
 - send a message to the RabbitMQ
 - will be processed by an "empty" OCR-worker
4. Failure/exception-handling (with layer-specific exceptions) implemented
5. Logging in remarkable/critical positions integrated
6. Prepare for the mid-term Code-Review

Assignment:

- Submit the ZIP-Archive containing the full project **on-time**
(see Moodle Course: „Submission – Sprint 3)
- MUST-HAVE Check Criteria:
 - No build error (docker compose build)
 - docker compose up starts all containers successfully
 - POST <http://localhost/>... some PDF-Document
will lead to a log-entry at the worker-service (for later processing in sprint 4)



Sprint 4: Worker Services (OCR, MinIO, ELK)

1. Create an additional applications for running the worker services
2. Tesseract for Ghostscript (or the like) integrated and working
3. Extend REST Server to store PDF document in MinIO
4. Implement the OCR-worker service to retrieve messages from the queue, fetch the original PDF-document from MinIO, and perform the OCR-recognition
5. Elasticsearch integrated in worker-service
6. Implement the indexing-worker to store the text-content (the former OCR-result) in Elasticsearch
7. Extend docker-compose.yml to run the MinIO, OCR-service, and ELK in containers

Assignment:

- Submit the ZIP-Archive containing the full project **on-time**
(see Moodle Course: „Submission – Sprint 4)
- MUST-HAVE Check Criteria:
 - No build error (docker compose build)
 - docker compose up starts all required containers
 - HelloWorld.pdf will be uploaded via Paperless Frontend <http://localhost/>
 - Search function executed in <http://localhost/> for term „Hello“ will result in showing up the HelloWorld.pdf document



Sprint 5: Generative AI-Integration

1. Extend docker-compose.yml to include configuration for GenAI service
2. Add a new GenAI-worker service
3. On document upload, after OCR is complete:
 - Send the extracted text to a GenAI API (Google Gemini)
 - Receive a summary as a response
4. Extend REST Server to store the summary in the database
5. Logging in critical positions is integrated
6. Exceptions and API failures are handled properly
7. Generate a smartphone app using AI coding tools.

Assignment:

- Submit the ZIP-Archive containing the full project **on-time**
(see Moodle Course: „Submission – Sprint 5)
- MUST-HAVE Check Criteria:
 - No build error
 - docker compose up starts all required containers
 - POST <http://localhost:8080/> ... some PDF-Document
 - summary is generated with GenAI
 - summary is stored in the database
 - smartphone app is available and working



Sprint 6: Integration-Test, Batch-Processing, Finalization

1. Show functionality of a full user-story/epic with an integration-test
2. Create an additional application for running the scheduled service for daily batch processing (input folder to process access-logs from external systems).
 - Define an appropriate XML format and provide sample files.
 - Store results in PostgreSQL database.
 - Schedule (e.g. daily at 01:00 AM), folder, filename-patterns are configurable.
 - Archive/remove processed files.
3. Project finalization
4. Prepare for the final code-review

Assignment:

- Submit the ZIP-Archive containing the full project **on-time**
(see Moodle Course: „Submission – Sprint 6)
- MUST-HAVE Check Criteria:
 - No build error
 - docker compose up starts all required containers
 - Provided integration-test (write an HOWTO in README.md) will be executed and should run successfully to the end.
 - The batch process must successfully read the sample XML file, process the data, and persist it in the database as demonstrated by relevant database queries.



Semester Project Grading



Grading

- 35% Continuous Sprint Submissions (code quality + completeness)
 - Incremental Points per Sprint
 - E.g. Sprint 3 = Sprint 1 Today, Sprint 2 Today, Sprint 3 Today
 - Continued option to improve
 - Hand-ins are graded and teams are asked for presentations
- 65% Code Reviews (code quality + knowledge + completeness)
 - 30% Mid-Review
 - 70% End-Review
- All parts have to be positive

Code Review

Code review is systematic examination ... of computer source code. It is intended to find and fix mistakes overlooked in the initial development phase, improving both the overall quality of software and the developers' skills.

Code Review Criterias

- Unit Tests, Code Coverage (> 70%)
- REST Service
- Queuing
- Business Layer & Logic
- Data Access Layer
- Entities & Entity Mapping
- Validation
- Exception Handling
- Logging
- Dependency Injection
- Service Agents
- Implementation of Use Cases